

Project Report: Smart MCQ Solver

Course: BSDA2001P: Introduction to Deep Learning and Generative AI Project

Name: Adrija De

Roll No: 23F2004791

1. Abstract / Executive Summary

This project addresses the Kaggle **Smart MCQ Solver Challenge**, where the objective is to predict the top three most likely correct answer options for each multiple-choice question, evaluated using the Mean Average Precision at 3 (MAP@3) metric. A complete deep learning workflow, beginning with exploratory data analysis (EDA), preprocessing, model development, training, evaluation and ensemble-based prediction has been followed.

Exploratory analysis identified several notable characteristics of the dataset. Besides a mild answer-position bias (option B tends to be correct more often), the analysis revealed structural biases in answer lengths (correct answer tends to be longer on average) as well as extensive overlap and duplication between the training and test sets. These observations were investigated before model training to better understand potential factors influencing model performance.

Three deep learning approaches were implemented and compared: a BiLSTM with an attention mechanism, a Transformer encoder trained from scratch, and a fine-tuned ELECTRA model. While the BiLSTM and Transformer models were trained using a custom vocabulary built from the competition data, the ELECTRA model leveraged a pretrained tokenizer and language model. Finally, predictions from multiple models were combined using a weighted ensemble to generate the competition submission.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge provides a training dataset containing 2,000 labelled multiple-choice questions and a test dataset containing 500 unlabelled questions. Each question consists of a prompt and five answer options (A-E). The objective is to predict the top three most probable answer choices for every test question, with submissions evaluated using Mean Average Precision at 3 (MAP@3).

2.2 Project Objectives

The objectives of this project are:

- Perform exploratory data analysis to understand the characteristics and potential biases present in the dataset.
- Design appropriate preprocessing and tokenization pipelines for both custom neural models and pretrained transformer models.
- Build and compare at least three qualitatively different modelling approaches (baseline, fine-tuned transformer, from-scratch transformer),
- Evaluate the strengths of different modelling approaches and combine their predictions using a weighted ensemble to improve overall performance.

- Gain practical experience with PyTorch, Hugging Face Transformers, Weights & Biases, and the end-to-end workflow involved in solving a real-world NLP competition.

2.3 Report Structure

Section 3 describes the dataset and exploratory data analysis. Section 4 discusses the preprocessing and tokenization strategies adopted for different models. Section 5 presents the architectures, training procedures, and implementation details of the BiLSTM, Transformer, and ELECTRA models, along with the ensemble strategy. Section 6 summarizes the experimental results and comparative analysis, while Section 7 concludes the report with key learnings, limitations, and possible future improvements.

3. Dataset & Preprocessing

3.1 Dataset Description

Item	Detail
Train / Test shape	2,000 rows × 8 columns / 500 rows × 7 columns
Columns	id, prompt, A, B, C, D, E, answer
Missing values	None
Submission format	id, prediction (top 3 predicted answer labels separated by spaces)

3.2 Exploratory Data Analysis

Before model development, exploratory data analysis (EDA) was performed to better understand the characteristics of the competition dataset and identify potential biases.

The analysis revealed several important observations:

- The distribution of correct answer labels is not perfectly uniform across the five options, indicating a mild positional bias in the dataset. Option B is the correct answer for 24.5% of the questions.

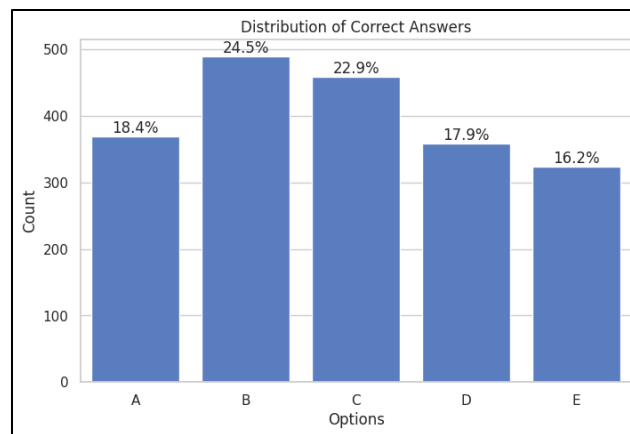


Fig 1: Distribution of Correct Answers

- The lengths of prompts and answer options were analyzed using both word count and character count. It was observed that 99th percentile of the total max length (prompt + longest option) is at 112 words. So 128 is chosen as max_seq_len in the code to cover >99% of sequences without truncation.

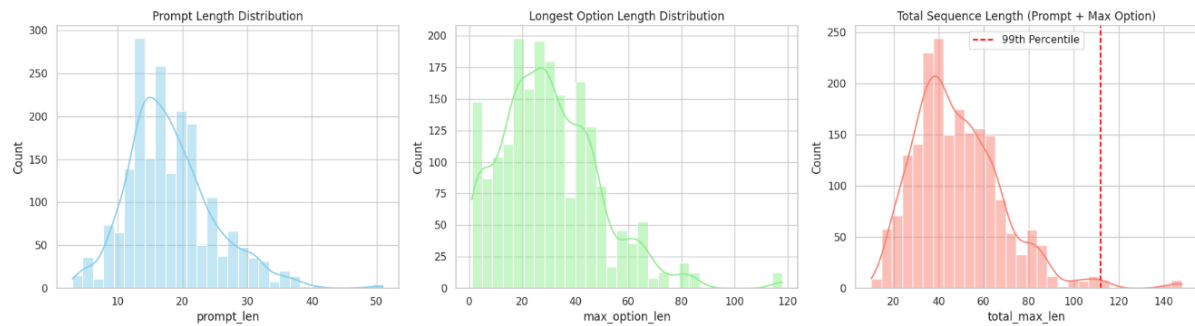


Fig 2: Some visualizations on Text Analysis

- The notebook investigated overlap between the training and test datasets using lexical similarity and duplicate detection techniques. This analysis suggested a significant amount of shared prefixes, suffixes, and near-duplicate questions between the two datasets:
 - Train set duplication: Of 2,000 training rows, only 964 (48.2%) represent unique questions after normalizing away instructional prefixes/suffixes. The remaining 1,442 rows (72.1%) are duplicates of another train row, with a single question repeating up to 11 times. This means the effective diversity of the training set is far smaller than its row count suggests.
 - Train/test overlap: 440 of 500 test rows (88.0%) have an exact normalized-prompt match somewhere in train. Only 60 rows (12.0%) contain genuinely unseen question content and even some of these may be near-duplicates that this exact-match normalization doesn't catch (e.g. reworded phrasing, reordered options). This overlap and duplication explains the data-leakage observed later while training.

These observations helped guide subsequent modelling decisions and provided useful context when interpreting model performance.

3.3 Data Preprocessing

- Missing values were not present in the dataset, so no imputation was required.
- For the BiLSTM and Transformer models, a custom vocabulary was constructed from the training data, and answer labels were mapped to integer class indices.
- For BiLSTM and Transformer models, each multiple-choice question was converted into individual question-option pairs so that every answer choice could be processed independently by the neural network.
- Text sequences were converted into integer token indices, padded or truncated to a fixed sequence length, and batched using custom PyTorch Dataset and DataLoader classes.
- For the ELECTRA model, preprocessing followed the Hugging Face multiple-choice format, where each question was paired with its five candidate answers before tokenization.

No explicit text cleaning such as stop-word removal or stemming was applied, allowing the models to learn directly from the original question text.

4. Tokenization Strategy

4.1 Tokenizer Selection

Different tokenization strategies were used for the different model architectures.

- The BiLSTM and Transformer-from-scratch models use a custom word-level vocabulary constructed from the competition dataset. Each unique token is assigned an integer index, with dedicated indices reserved for padding and unknown words.
- The ELECTRA model uses the pretrained tokenizer associated with the selected ELECTRA checkpoint. Using the pretrained tokenizer ensures consistency with the language representations learned during ELECTRA pretraining.

4.2 Implementation Details

- For the BiLSTM and Transformer-from-scratch, a custom word-level tokenizer (SimpleVocab) was built using all prompts and answer options from the training set. Text was converted to lowercase, punctuation was removed, and whitespace tokenization was applied. Words occurring at least twice ($\text{min_freq} = 2$) were added to the vocabulary, while <PAD> and <UNK> tokens handled padding and unseen words. Each question was concatenated with one answer option, converted to token IDs, and either truncated or padded to a fixed length of 128 tokens, producing a 5×128 input tensor for every multiple-choice question.
- For the ELECTRA model, tokenization was performed using Hugging Face's AutoTokenizer, which automatically loads the tokenizer corresponding to the selected pretrained ELECTRA checkpoint. Unlike the custom tokenizer, AutoTokenizer uses the same pretrained subword vocabulary and tokenization rules as the language model, ensuring compatibility between the encoded inputs and the model weights. Each question was paired with its five answer options and tokenized in a single batched call, with truncation and padding applied automatically. The tokenizer also generated attention masks (and token type IDs, where applicable) and returned the encoded inputs directly as PyTorch tensors (`return_tensors="pt"`), enabling efficient training and inference with the multiple-choice ELECTRA model.

4.3 Experimental Comparison

Before finalizing the custom vocabulary tokenizer, pretrained Word2Vec embeddings were also experimented with for the BiLSTM and Transformer-from-scratch models. However, the custom vocabulary consistently produced better scores on the public leaderboard. Given the relatively small size and domain-specific nature of the competition dataset and the overlap in training and test datasets, learning task-specific embeddings jointly with the model proved more effective than using fixed pretrained Word2Vec representations. Consequently, the custom vocabulary tokenizer was adopted for both scratch models, while the ELECTRA model retained its pretrained AutoTokenizer to preserve compatibility with the pretrained transformer weights.

5. Modeling & Experimentation

5.1. Model 1: Fine-Tuned Transformer Model – ELECTRA

- **Architecture:** The first model is a fine-tuned **ELECTRA** model implemented using Hugging Face's AutoModelForMultipleChoice. ELECTRA is an encoder-only Transformer architecture proposed by Clark et al. (2020) that is pretrained using a replaced token detection objective rather than masked language modelling, making it more sample-efficient than BERT. For this task, each

question is paired independently with its five answer options, and the shared transformer encoder produces a score for each pair. These scores are converted into probabilities using a softmax layer to predict the most likely answer.

- **Salient Points:** ELECTRA was selected because its encoder-only architecture is well suited for multiple-choice classification, where the objective is to compare candidate answers rather than generate text. Its self-attention mechanism captures contextual relationships between the question and each answer option, while pretraining on large text corpora provides rich language representations that can be effectively adapted to the competition dataset through fine-tuning.
- **Fine-Tuning Strategy:** The pretrained ELECTRA model was fine-tuned end-to-end using the Hugging Face Transformers framework, with all layers kept trainable. Training was performed using the AdamW optimizer with a learning rate of 2×10^{-5} a cosine learning rate scheduler with warm-up ratio of 0.1, and 5 training epochs. Full fine-tuning was preferred over freezing the encoder layers to allow the pretrained representations to adapt to the domain-specific characteristics of the dataset. Although five epochs is relatively small, it was selected to balance effective adaptation with the risk of overfitting on the limited training data. Among the individual models evaluated, this configuration achieved the best standalone performance on the Kaggle leaderboard.

5.2. Model 2: Custom Deep Learning Model – BiLSTM with Attention

- **Architecture:** A custom Attention-Based Bidirectional Long Short-Term Memory (BiLSTM) network is implemented in PyTorch. Each multiple-choice question is represented as five (*prompt, option*) pairs, which are first converted into integer token IDs using the custom vocabulary and mapped to dense vector representations through a trainable embedding layer. The embedded sequences are then passed through a two-layer bidirectional LSTM, enabling the model to capture contextual information from both forward and backward directions. An attention layer computes a weighted combination of the hidden states to produce a fixed-length context vector, which is passed through a fully connected classification network to generate one score for each answer option. The five scores are reshaped into a probability distribution over the candidate answers using a softmax operation.

The architecture of this custom model has been shown in the form of a diagram in Fig 3.

- **Salient Points:** A BiLSTM was selected because understanding the relationship between a question and its answer option often requires contextual information from both preceding and succeeding words. The bidirectional architecture captures dependencies in both directions, while the attention mechanism enables the model to focus on the most informative parts of the sequence instead of relying only on the final hidden state. A two-layer LSTM with dropout was used to improve the model's representational capacity while reducing overfitting, and the fully connected classification head learns to score each answer option independently before selecting the most probable response.

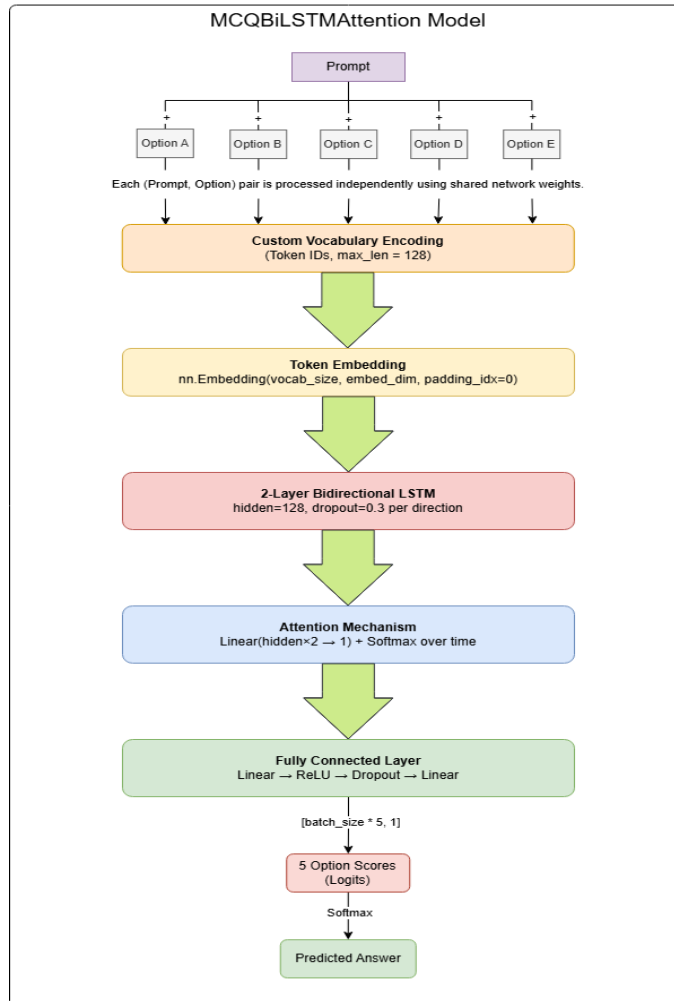


Fig 3: Diagram showing the architecture of the custom BiLSTM model with Attention Mechanism

5.3. Model 3: Custom Encoder-Only Transformer

- Architecture:** The third model is a lightweight Transformer encoder built entirely from scratch using PyTorch. Similar to the BiLSTM model, each multiple-choice question is converted into five (*prompt*, *option*) pairs and encoded using the custom vocabulary. The token IDs are first mapped to trainable word embeddings, which are combined with sinusoidal positional encodings to preserve word order. The resulting embeddings are passed through a two-layer Transformer encoder consisting of 4-head self-attention, feed-forward networks with GELU activation, residual connections, layer normalization, and dropout. Padding masks ensure that padded tokens do not contribute to the attention computation. The encoder outputs are aggregated using mean pooling over the valid (non-padding) tokens, followed by Layer Normalization and a fully connected classification head that produces one score for each answer option. The five scores are reshaped into logits and converted into probabilities using a softmax operation.

The architecture of this custom transformer model has been shown in the form of a diagram in Fig4.

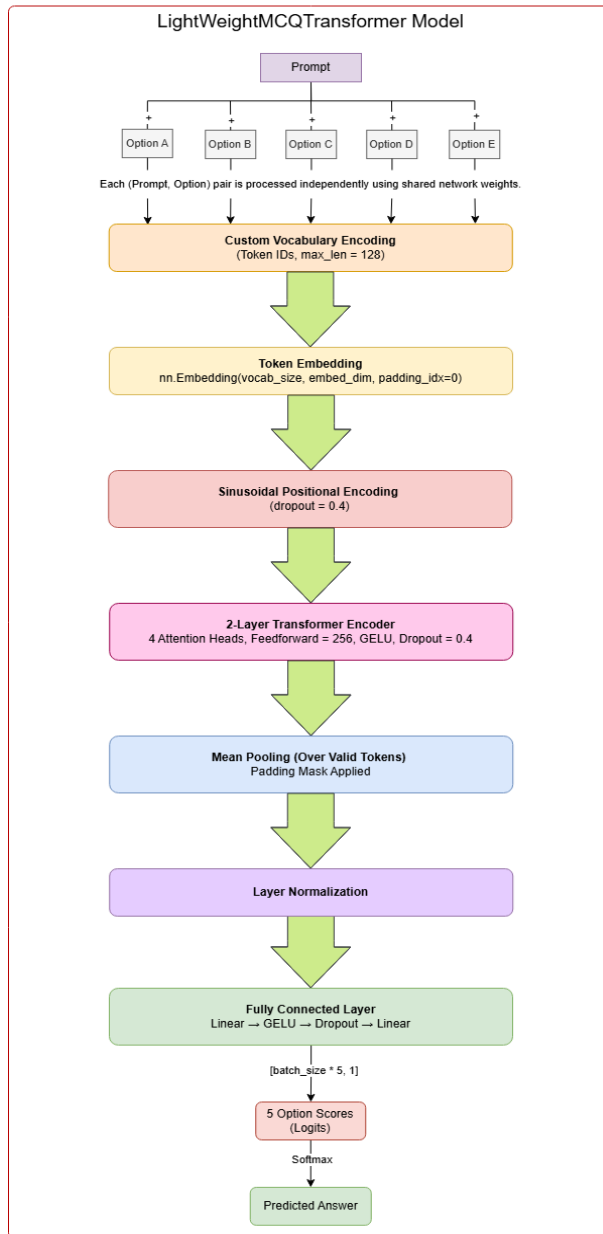


Fig 4: Diagram showing the architecture of the custom Transformer model with Positional Encoding

- Salient Points:** Unlike the pretrained ELECTRA model, this Transformer was trained entirely from scratch using a custom vocabulary and randomly initialized weights. The self-attention mechanism enables every token to attend to all other tokens in the sequence, allowing the model to capture long-range dependencies between the question and its candidate answers without relying on recurrent computations. Sinusoidal positional encoding was incorporated to retain sequence order, while multi-head attention allowed the model to learn different contextual relationships simultaneously. Mean pooling over valid tokens produced a fixed-length sequence representation while avoiding the influence of padded positions. The lightweight two-layer architecture was chosen to balance representational capacity with the relatively small size of the training dataset, reducing computational complexity and the risk of overfitting.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

Model performance was evaluated using multiple metrics to assess both classification accuracy and ranking quality. The primary evaluation metric specified by the Kaggle competition was Mean Average Precision at 3 (MAP@3), which measures whether the correct answer appears within the top three predicted options while assigning higher rewards to correctly ranked predictions. Since the competition requires predicting the three most likely answers rather than a single class label, MAP@3 was considered the most important performance metric. In addition to MAP@3, accuracy and macro F1-score were computed on the validation set to evaluate classification performance across all answer classes. During training, both training loss and validation loss were monitored to evaluate convergence and identify potential overfitting.

6.2 Training Performance

Training loss, validation loss, validation MAP@3, validation accuracy, and validation macro F1-score were recorded after every epoch in each model to analyse convergence and compare model performance.

Attention-based BiLSTM:

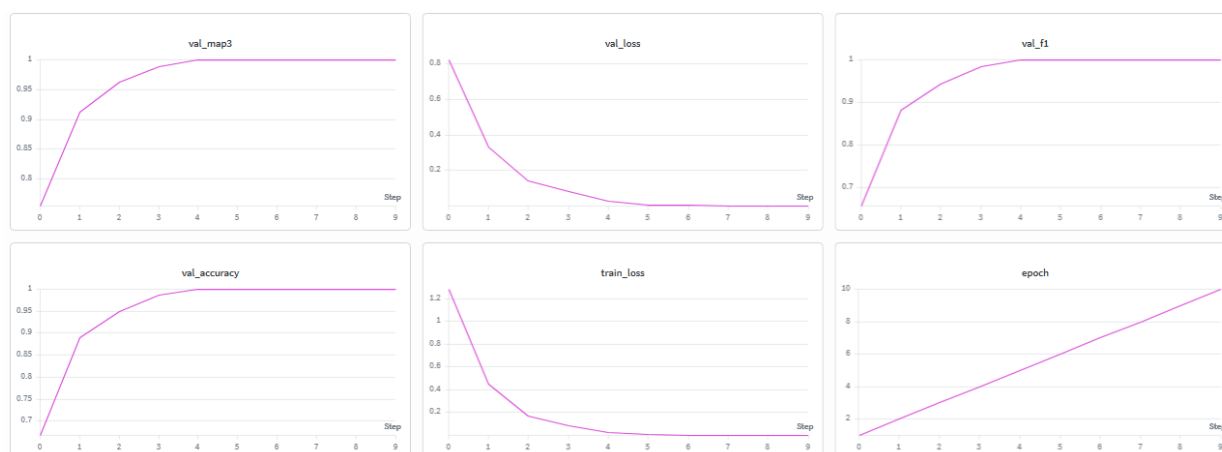


Fig 5: Training curves for BiLSTM

The BiLSTM exhibited rapid convergence, with validation MAP@3 improving from 0.7550 in the first epoch to 1.0000 by the fifth epoch. Both training and validation losses decreased steadily, while validation accuracy and macro F1-score reached 1.0000 and remained stable for the remaining epochs. These results indicate that the model fit the training and validation data extremely well. Although such behaviour would typically suggest overfitting, the exploratory data analysis revealed substantial overlap and duplication between the training and test datasets. As a result, memorizing recurring patterns did not significantly harm generalization and, in many cases, contributed to improved leaderboard performance. Nevertheless, the lower Kaggle score compared to its perfect validation performance indicates that some degree of overfitting was still present.

Fine-tuned ELECTRA:

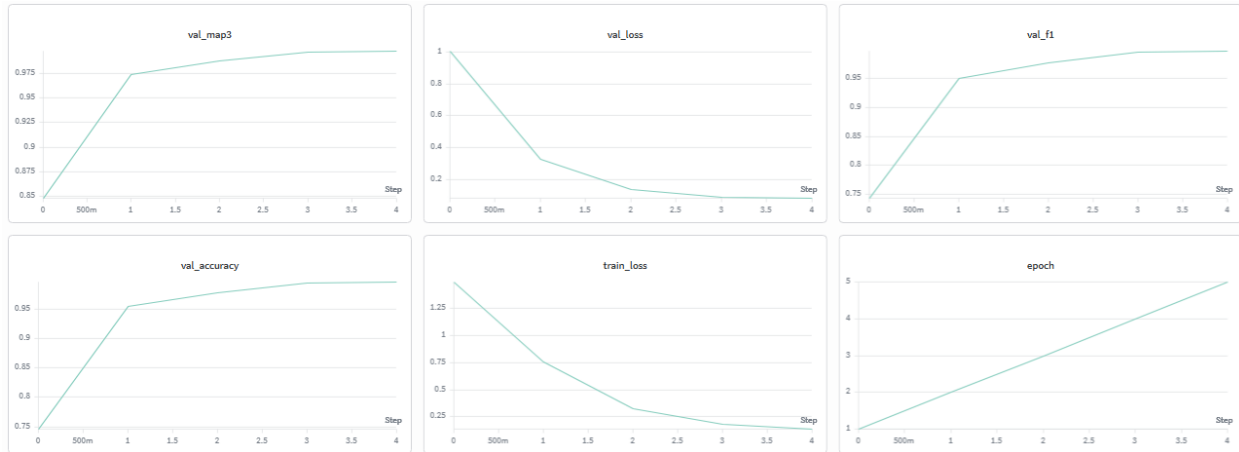


Fig 6: Training curves for electra-base

The ELECTRA model showed stable and consistent learning throughout the five training epochs. Validation MAP@3 increased from 0.8480 to 0.9980, while validation loss decreased continuously from 1.0066 to 0.0814. Compared to the BiLSTM, ELECTRA achieved slightly lower validation metrics but demonstrated better generalization on the unseen Kaggle test set, resulting in the highest standalone leaderboard score among the three models.

Transformer with Positional Encoding

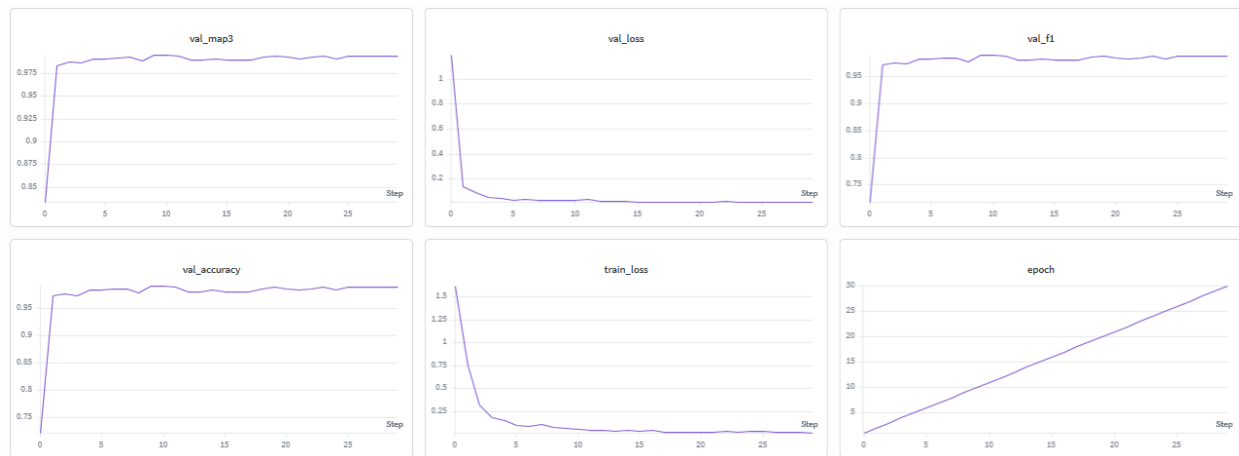


Fig 7: Training curves for Transformer

The Transformer model converged rapidly during the initial epochs, with validation MAP@3 reaching approximately 0.99 within the first few epochs. After this point, the validation metrics fluctuated only slightly while the training loss continued to decrease, indicating that the model had largely converged. Despite achieving strong validation performance, its leaderboard score was lower than both the BiLSTM and ELECTRA models, suggesting that training a transformer entirely from scratch on a relatively small dataset limited its ability to generalize compared to the pretrained ELECTRA model.

6.3 Comparative Report

The following table summarizes the performance of the three standalone models together with the final ensemble:

Model	Best Validation MAP@3	Best Validation Accuracy	Best Validation Macro F1	Kaggle Leaderboard Score
Attention-Based BiLSTM	1.0000	1.0000	1.0000	0.75685
Fine-Tuned ELECTRA	0.9980	0.9960	0.9962	0.75893
Transformer (Scratch)	0.9950	0.9900	0.9899	0.74646
BiLSTM + ELECTRA Ensemble	—	—	—	0.75976

While the BiLSTM achieved perfect validation metrics, the fine-tuned ELECTRA model obtained the highest standalone Kaggle leaderboard score. This suggests that pretrained contextual representations generalized better overall. However, the extensive overlap and duplication observed between the training and test sets reduced the typical penalty associated with overfitting, allowing highly fitted models such as the BiLSTM to remain competitive on the leaderboard. The Transformer trained entirely from scratch also achieved high validation performance; however, its leaderboard score suggests that the limited size of the training dataset restricted the effectiveness of learning transformer representations from scratch.

To exploit the complementary strengths of different architectures, predictions from the BiLSTM and ELECTRA models were combined using a weighted ensemble. The ensemble achieved the highest Kaggle leaderboard score of **0.75976**, outperforming all individual models. This demonstrates that combining predictions from models with different learning characteristics can improve robustness and generalization beyond what is achievable with a single architecture.

6.4 Kaggle Performance

The final competition submission was generated using a weighted ensemble of the Attention-Based BiLSTM and fine-tuned ELECTRA models. This approach achieved a final Kaggle MAP@3 score of 0.75976, outperforming all standalone models evaluated during the project.

Final Kaggle Score: 0.75976

Final Leaderboard Rank: 207 out of 1531 participants

7. Conclusion & Future Work

7.1 Key Learnings

- Comparing three fundamentally different architectures (Attention-Based BiLSTM, Transformer trained from scratch, and fine-tuned ELECTRA) demonstrated the trade-offs between learning representations from scratch and leveraging large-scale pretrained language models. While the pretrained ELECTRA model achieved the best standalone leaderboard score, the scratch models remained competitive despite having significantly fewer prior language representations.

- Careful exploratory data analysis proved to be as important as model selection. The discovery of substantial overlap and duplication between the training and test datasets helped explain the unusually high validation scores and the comparatively modest leaderboard performance.
- The project highlighted that validation performance should not always be interpreted as a measure of real-world generalization. Because of duplicate and overlapping samples, allowing the scratch models to fit the training data more aggressively often improved competition performance rather than degrading it, although excessive memorization still reduced generalization on unseen examples.
- Hyperparameter tuning, including learning rate, dropout, weight decay, scheduler selection, and training duration, played a significant role in stabilizing training and obtaining competitive performance across different architectures.
- Combining complementary models through weighted ensembling produced the best leaderboard score, reinforcing that models with different learning behaviours can compensate for one another's prediction errors.

7.2 Challenges Faced

- Limited labelled data: With only 2,000 training questions, the scratch models were particularly susceptible to overfitting, requiring careful tuning of regularization techniques and training schedules.
- Dataset overlap and duplicate questions: Extensive overlap between the training and test datasets complicated performance evaluation. While memorization improved leaderboard performance in many cases, it also made validation metrics less representative of true generalization.
- Model selection and comparison: Developing three fundamentally different architectures while ensuring a consistent preprocessing pipeline and evaluation methodology required careful experimental design.
- Balancing accuracy and computational cost: Fine-tuning ELECTRA provided the strongest standalone performance but required significantly longer training times and greater computational resources than the custom models.

7.3 Areas for Improvement

- Retrieval-Augmented Generation (RAG): Instead of relying solely on information learned during training, a retrieval component could fetch relevant external knowledge (e.g., Wikipedia or a curated knowledge base) for each question before prediction. This would allow the model to answer factual questions using retrieved evidence rather than memorized information and could improve generalization to previously unseen questions.
- Ranking-aware loss functions: Since the competition is evaluated using MAP@3, future work could investigate ranking-based objectives instead of relying solely on cross-entropy loss, enabling the model to optimize directly for the competition metric.
- Data Augmentation: Introduce data augmentation techniques, such as paraphrasing questions or shuffling answer option order while updating the corresponding labels, to increase training diversity and reduce reliance on duplicated samples.

- Domain-specific pretraining: Rather than simply adopting a larger pretrained model, further pretraining on educational, scientific, or general knowledge corpora related to the competition domain could improve downstream multiple-choice reasoning performance before task-specific fine-tuning.

8. References

1. Clark, K., Luong, M. T., Le, Q. V., & Manning, C. D. (2020). ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. Proceedings of the International Conference on Learning Representations (ICLR).
2. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS), 30.
3. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.
4. Hugging Face Transformers Documentation. <https://huggingface.co/docs/transformers>
5. Hugging Face google/electra-base-discriminator <https://huggingface.co/google/electra-base-discriminator>